



ELISA

Enabling **Linux** in
Safety Applications

Safety Software Supply Chain: SPDX-Reqs in the Linux Kernel

OpenChain Automotive Work Group Workshop | Online on 2025-11-14, 9:00 ~ 12:00 CET Brussels/Europe

Philipp Ahmann, ETAS GmbH



Linux Kernel Requirements Proposal

Activity started within ELISA project.

Brought to Linux Plumbers conference in 2024

Continued via LKML

Will be brought to Plumbers in 2025 again.

- Prototyping Linux Kernel Requirements
- Prototyping the automation to check patch sets against requirements & vice-versa
- Add formal specification description to Kernel Documentation

Considered parameter

- `SPDX-Req-ID`
- `SPDX-Req-End`
- `SPDX-Req-Ref`
- `SPDX-Req-HKey`
- `SPDX-Req-Child`
- `SPDX-Req-Sys`
- `SPDX-Req-Text`
- `SPDX-Req-Note`

Prototyping Linux Kernel Requirements

```
1498  /**
1499   * SPDX-Req-ID: [TODO automatically generate it]
1500   * SPDX-Req-Text:
1501   * trace_set_clr_event - enable or disable an event within a system
1502   * @system: system name (NULL for any system)
1503   * @event: event name (NULL for all events, within system)
1504   * @set: 1 to enable, 0 to disable (any other value is invalid)
1505   *
1506   * This is a way for other parts of the kernel to enable or disable
1507   * event recording.
1508   *
1509   * sequence of events:
1510   * 1) retrieve the global tracer
1511   * 2) locks the global event_mutex
1512   * 3) invokes __ftrace_set_clr_event_nolock
1513   * 4) unlocks the global event_mutex
1514   *
1515   * Returns 0 on success, -ENODEV if the global tracer cannot be retrieved,
1516   * -EINVAL if the parameters do not match any registered events, any other
1517   * error condition returned by __ftrace_set_clr_event_nolock
1518   */
1519  int trace_set_clr_event(const char *system, const char *event, int set)
1520  {
```

- Currently prototyping requirements for some functions in tracing subsystem.
- Requirements shall be:
 - Testable
 - Maintainable inline within the source code
 - Compatible with pre-existing Kernel Doc.
 - Hierarchically traceable
- The main challenge is identifying the main design elements to be documented starting from the pre-existing code

Prototyping the Automation to Check Patch Sets

✓ `scripts/reqs/idgen.py`: Add script for SPDX-Req-ID management"

This script scans and processes all `.c` and `.h` files within a directory tree.

It performs two main tasks:

- * Preprocessing: Detects existing SPDX-Req-ID entries, updating a global map to track the highest progressive ID per file hash.
- * ID Assignment: Updates or assigns new SPDX-Req-ID identifiers where missing, based on the file's hash and the next available progressive ID.

The script ensures efficient directory traversal by maintaining a single file system scan and processes files in place, emitting warnings for invalid or mismatched IDs.

Signed-off-by: Alessandro Carminati <acarmina@redhat.com>



alessandrocarminati committed 5 days ago

A script to automate the generation of Requirements' IDs (`SPDX-Req-ID`) is in progress.

The goal is to generate a unique one that cannot change along the life of the requirements

"`SPDX-Req-HKey`" will instead be used to flag if, code or requirement's text changes, so that the requirement will be reviewed against the code (and vice versa).

"`SPDX-Req-HKey`" hashes are produced based on the following criteria:

- PROJECT: The name of the project (e.g. linux)
- FILE_PATH: The file the code resides in, relative to the root of the project repository.
- INSTANCE: The requirement template instance, minus tags with hash strings.
- CODE: The code that the SPDX-Req applies to.

"`SPDX-Req-ID`" is the very first "`SPDX-Req-HKey`" generated

Some links for further reading...

Requirements work base:

- [ELISA WIP Draft document: Linux Kernel Requirements Template](#)
- [PR: scripts/reqs/idgen.py: Add script for SPDX-Req-ID management](#)
- https://github.com/elisa-tech/linux/tree/linux_requirements_wip

Formal testable code specification work:

- [\[RFC PATCH v2 0/3\] Add testable code specifications](#)
- [\[RFC v2 PATCH 2/3\] /dev/mem: Add initial documentation of memory_open\(\) and mem_fops](#)
- [Plumbers: Adding Testable Code Specifications in the Linux Kernel](#)

Thank you

